Task-8: Employee Management module using Spring Core.

Aim: To develop a simple **Employee Management Module** that demonstrates the concepts of **Inversion of Control (IoC)** and **Dependency Injection (DI)** using a Node.js-based architecture. The system allows users to **add and view employee details**, with data stored in memory and managed through a custom container.

Algorithm:

1. Start the application.
2. Create an Employee class with fields:
    - id, name, role
3. Create an EmployeeService to:
    - Store employee data in a list
    - Add new employee
    - Retrieve all employees
4. Create a container (BeanFactory) to:
    - Create EmployeeService object
    - Return the same object whenever needed
5. Initialize the server using Node.js and Express.
6. Inject EmployeeService from the container into the server.
7. Create API endpoints:
    - POST /add → Add employee
    - GET /employees → Get all employees
8. Design a simple HTML page:
    - Input fields for id, name, role
    - Button to add employee
    - Section to display employee list
9. Use JavaScript:
    - Send data to server using fetch()
    - Get employee list from server
    - Display data on the page
10. When user adds employee:

- Store data in memory
- Update employee list

11. Stop the application.

Program:

Index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Employee Management</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
<h2>Add Employee</h2>
<input type="text" id="id" placeholder="ID">
<input type="text" id="name" placeholder="Name">
<input type="text" id="role" placeholder="Role">
<button onclick="addEmployee()">Add</button>
<h2>Employee List</h2>
<ul id="list"></ul>
<script src="script.js"></script>
</body>
</html>
```

Style.css

```
body {
   font-family: Arial;
   padding: 20px;
}

input {
   margin: 5px;
   padding: 5px;
}
```

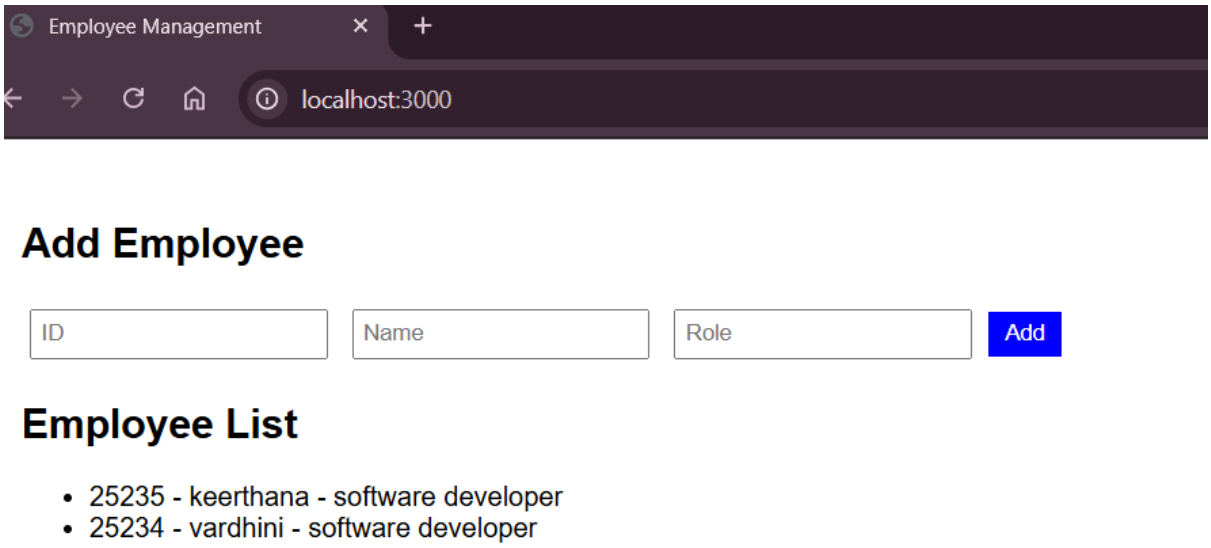
```
button {
    padding: 5px 10px;
    background: blue;
    color: white;
    border: none;
}
```

Script.js

```
function addEmployee() {
    const id = document.getElementById("id").value.trim();
    const name = document.getElementById("name").value.trim();
    const role = document.getElementById("role").value.trim();
    // Basic validation
    if (!id || !name || !role) {
        alert("Please fill all fields");
        return;
    }
    fetch("/add", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({ id, name, role })
    })
    .then(res => res.text())
    .then(msg => {
        alert(msg);
        // Clear inputs after adding
        document.getElementById("id").value = "";
        document.getElementById("name").value = "";
        document.getElementById("role").value = "";
```

```
        loadEmployees(); // Refresh list
    })
    .catch(err => {
        console.error("Error adding employee:", err);
    });
}
function loadEmployees() {
    fetch("/employees")
        .then(res => res.json())
        .then(data => {
            const list = document.getElementById("list");
            list.innerHTML = "";
            // Handle empty list
            if (!data || data.length === 0) {
                list.innerHTML = "<li>No employees found</li>";
                return;
            }
            data.forEach(emp => {
                const li = document.createElement("li");
                li.textContent = `${emp.id} - ${emp.name} - ${emp.role}`;
                list.appendChild(li);
            });
        })
        .catch(err => {
            console.error("Error loading employees:", err);
        });
}
// Load employees when page loads
window.onload = loadEmployees;
```

output:



Result: The Employee Management module was successfully developed using Node.js. Employee details such as ID, Name, and Role are added through the web interface and stored in memory. The system successfully adds and displays employee details using Node.js with IoC and Dependency Injection.